

TRABAJO GRUPAL 1: ECUACIÓN DE CALOR 2D PROGRAMACIÓN PARALELA Y CONCURRENTE

Docente: Jaime Salvador M.

Fecha: 2026-07-06

Duración: 2 semanas, **Puntos:** 20 puntos (rúbrica adjunta)

Nombre:
Firma:

1. Objetivo

Implementar en C/C++ la resolución numérica de la **ecuación de calor 2D** por diferencias finitas (esquema de 5 puntos, Jacobi o Gauss-Seidel):

Backend	backend	Estándares del sílabo usados
1. Serial de referencia	serial	(ninguno)
2. Vectorización SIMD	simd	Vectorización / SIMD
3. OpenMP multinúcleo	openmp	OpenMP bucles y vectorización SIMD
4. MPI	mpi	MPI comunicación point-to-point + comunicación colectiva

La aplicación debe **renderizar el campo escalar** $u(x, y)$ **en una ventana SFML** mediante un mapa de calor (paleta de colores utilizada en el dibujo de fractales `palette.h/cpp`), permitir avanzar paso a paso o en modo continuo, y mostrar en pantalla el **backend activo**, **iteración actual**, **residuo L_2** y **MFLOPS estimados**.

2. Aplicación: ecuación de calor 2D

Ecuación:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \quad (x, y) \in \Omega = (0, L_x) \times (0, L_y), \quad t > 0$$

Discretización por diferencias finitas en grilla uniforme $N_x \times N_y$ con paso $h = L_x/N_x = L_y/N_y$, paso temporal Δt , y condición CFL $\Delta t \leq h^2/(4\alpha)$. Esquema de 5 puntos:

$$u_{\text{new}}[i, j] = u[i, j] + \frac{\alpha \cdot \Delta t}{h^2} \cdot (u[i+1, j] + u[i-1, j] + u[i, j+1] + u[i, j-1] - 4 \cdot u[i, j])$$

Condiciones de contorno (Dirichlet fijas):

- Borde superior $u(x, L_y) = 100.0$ (fuente caliente)
- Los otros tres bordes $u = 0.0$
- Estado inicial $u(x, y, 0) = 0.0$ en el interior

Criterio de parada: número máximo de iteraciones (por defecto 1000) o residuo L_2 por debajo de `--tol` (por defecto 1×10^{-4}):

$$\|r\|_2 = \sqrt{\sum (u_{\text{new}} - u_{\text{old}})^2 / (N_x \cdot N_y)}$$

2.1 Valores por defecto de la simulación

Los flags de la línea de comandos toman los siguientes valores por omisión (alineados con `struct Args` del código de referencia):

Parámetro	Símbolo	Default	Significado
Tamaño de grilla en x	N_x	1024	Celdas en el eje horizontal
Tamaño de grilla en y	N_y	1024	Celdas en el eje vertical
Longitud física en x	L_x	1.0	Dominio espacial horizontal
Longitud física en y	L_y	1.0	Dominio espacial vertical
Paso espacial en x	h_x	$L_x/N_x \approx 9.77 \times 10^{-4}$	Discretización de la grilla
Paso espacial en y	h_y	$L_y/N_y \approx 9.77 \times 10^{-4}$	Discretización de la grilla
Cuadrado del paso	h^2	$h_x \cdot h_y \approx 9.54 \times 10^{-7}$	Aparece en el denominador del stencil
Difusividad térmica	α	0.25	Coefficiente de la ecuación
Paso temporal	Δt	5.0e-7	Adelanto en el tiempo
Parámetro de estabilidad	r	$\alpha \Delta t / h^2 \approx 0.131$	Debe cumplir $r \leq 0.25$ (CFL 2D)
Iteraciones máximas	--iter	1000	Límite del solver
Tolerancia de residuo	--tol	1.0e-4	Criterio de parada por $\ r\ _2$
Borde superior (Dirichlet)	$u(x, L_y)$	100.0	Fuente caliente fija
Estado inicial	$u(x, y, 0)$	0.0	Interior en frío
Tiempo físico simulado	T	$N_{\text{iter}} \cdot \Delta t = 5 \times 10^{-4}$	Duración total corrida
Tiempo característico	L^2/α	4.0	Escala natural de difusión
Fracción recorrida	$T/(L^2/\alpha)$	$\approx 0.012\%$	Indicador de avance de la difusión

Cómputo de h : con $N_x = N_y = 1024$ y $L_x = L_y = 1.0$, se tiene $h = 1/1024$. Si se reduce N_x a 128 (mismo L_x), h pasa a $1/128$ y se puede usar un Δt del orden de 10^{-5} manteniendo $r < 0.25$, lo cual produce una evolución más visible en menos iteraciones.

3. Software obligatorio

Componente	Requisito	Notas
Lenguaje	C/C++	GCC ≥ 14
Build	CMake ≥ 3.16	Debe compilar en Windows
Render	SFML 2.6+ (módulos graphics, window)	
OpenMP	≥ 4.5	#pragma omp parallel (paralelización manual)
MPI	Intel MPI	

4. Requerimientos funcionales

4.1 Render SFML

- Ventana 1600×900 px.
- Cada celda se mapea a un píxel del mapa de calor.
- Overlay de texto en la esquina superior izquierda (usar `arial.ttf`) con información del renderizado.
- Overlay de texto en la esquina inferior izquierda (usar `arial.ttf`) con menú de opciones.

5. Comandos de verificación

El grupo debe asegurarse de que el programa compile en una máquina Windows con GCC, Intel MPI y SFML instalados:

6. Rúbrica (20 puntos)

#	Criterio	Pts	Temas sílabo
1	Backend serial correcto, funcional	3	Vectorización
2	Backend simd con intrínsecas presentes y funcional	3	SIMD
3	Backend openmp correcto, OMP_NUM_THREADS configurable	4	OpenMP regiones paralelas
4	Backend MPI con comunicación punto a punto y colectiva	6	MPI point-to-point + colectivas

#	Criterio	Pts	Temas sílabo
5	Informe con análisis speedup/eficiencia, screencast ≤ 3 min, código limpio (CMake estándar, sin warnings, sin código muerto)	4	Patrones paralelos: reducción
	Total	20	

7. Política de originalidad y uso de IA

- El ejemplo fractal-julia realizado en clase es una **referencia**, no plantilla. No se permite copiar bloques de más de 30 líneas de esos ejemplos sin reescritura sustancial.
- Cualquier copia entre grupos (o de soluciones encontradas en línea) será calificada con **0**.
- En caso de detectarse proyectos generados íntegramente con IA generativa, recibirán la nota de **0**.
- El código entregado debe seguir la estructura del ejemplo realizado en clases.